

14.7 节练习

练习 14.30: 为你的 `StrBlobPtr` 类和在 12.1.6 节练习 12.22 (第 423 页) 中定义的 `ConstStrBlobPtr` 类分别添加解引用运算符和箭头运算符。注意: 因为 `ConstStrBlobPtr` 的数据成员指向 `const vector`, 所以 `ConstStrBlobPtr` 中的运算符必须返回常量引用。

练习 14.31: 我们的 `StrBlobPtr` 类没有定义拷贝构造函数、赋值运算符及析构函数, 为什么?

练习 14.32: 定义一个类令其含有指向 `StrBlobPtr` 对象的指针, 为这个类定义重载的箭头运算符。



14.8 函数调用运算符

571

如果类重载了函数调用运算符, 则我们可以像使用函数一样使用该类的对象。因为这样的类同时也能存储状态, 所以与普通函数相比它们更加灵活。

举个简单的例子, 下面这个名为 `absInt` 的 `struct` 含有一个调用运算符, 该运算符负责返回其参数的绝对值:

```
struct absInt {
    int operator()(int val) const {
        return val < 0 ? -val : val;
    }
};
```

这个类只定义了一种操作: 函数调用运算符, 它负责接受一个 `int` 类型的实参, 然后返回该实参的绝对值。

我们使用调用运算符的方式是令一个 `absInt` 对象作用于一个实参列表, 这一过程看起来非常像调用函数的过程:

```
int i = -42;
absInt absObj;                // 含有函数调用运算符的对象
int ui = absObj(i);            // 将 i 传递给 absObj.operator()
```

即使 `absObj` 只是一个对象而非函数, 我们也能“调用”该对象。调用对象实际上是在运行重载的调用运算符。在此例中, 该运算符接受一个 `int` 值并返回其绝对值。



函数调用运算符必须是成员函数。一个类可以定义多个不同版本的调用运算符, 相互之间应该在参数数量或类型上有所区别。

如果类定义了调用运算符, 则该类的对象称作**函数对象** (function object)。因为可以调用这种对象, 所以我们说这些对象的“行为像函数一样”。

含有状态的函数对象类

和其他类一样, 函数对象类除了 `operator()` 之外也可以包含其他成员。函数对象类通常含有一些数据成员, 这些成员被用于定制调用运算符中的操作。

举个例子, 我们将定义一个打印 `string` 实参内容的类。默认情况下, 我们的类会将

内容写入到 `cout` 中，每个 `string` 之间以空格隔开。同时也允许类的用户提供其他可写入的流及其他分隔符。我们将该类定义如下：

```
class PrintString {
public:
    PrintString(ostream &o = cout, char c = ' '):
        os(o), sep(c) { }
    void operator()(const string &s) const { os << s << sep; }
private:
    ostream &os;           // 用于写入的目的流
    char sep;              // 用于将不同输出隔开的字符
};
```

我们的类有一个构造函数，它接受一个输出流的引用以及一个用于分隔的字符，这两个形参的默认实参（参见 6.5.1 节，第 211 页）分别是 `cout` 和空格。之后的函数调用运算符使用这些成员协助其打印给定的 `string`。

◀ 572

当定义 `PrintString` 的对象时，对于分隔符及输出流既可以使用默认值也可以提供我们自己的值：

```
PrintString printer;           // 使用默认值，打印到 cout
printer(s);                   // 在 cout 中打印 s，后面跟一个空格
PrintString errors(cerr, '\n');
errors(s);                    // 在 cerr 中打印 s，后面跟一个换行符
```

函数对象常常作为泛型算法的实参。例如，可以使用标准库 `for_each` 算法（参见 10.3.2 节，第 348 页）和我们自己的 `PrintString` 类来打印容器的内容：

```
for_each(vs.begin(), vs.end(), PrintString(cerr, '\n'));
```

`for_each` 的第三个实参是类型 `PrintString` 的一个临时对象，其中我们用 `cerr` 和换行符初始化了该对象。当程序调用 `for_each` 时，将会把 `vs` 中的每个元素依次打印到 `cerr` 中，元素之间以换行符分隔。

14.8 节练习

练习 14.33: 一个重载的函数调用运算符应该接受几个运算对象？

练习 14.34: 定义一个函数对象类，令其执行 `if-then-else` 的操作：该类的调用运算符接受三个形参，它首先检查第一个形参，如果成功返回第二个形参的值；如果不成功返回第三个形参的值。

练习 14.35: 编写一个类似于 `PrintString` 的类，令其从 `istream` 中读取一行输入，然后返回一个表示我们所读内容的 `string`。如果读取失败，返回空 `string`。

练习 14.36: 使用前一个练习定义的类读取标准输入，将每一行保存为 `vector` 的一个元素。

练习 14.37: 编写一个类令其检查两个值是否相等。使用该对象及标准库算法编写程序，令其替换某个序列中具有给定值的所有实例。

14.8.1 `lambda` 是函数对象

在前一节中，我们使用一个 `PrintString` 对象作为调用 `for_each` 的实参，这一

用法类似于我们在 10.3.2 节（第 346 页）中编写的使用 `lambda` 表达式的程序。当我们编写了一个 `lambda` 后，编译器将该表达式翻译成一个未命名类的未命名对象（参见 10.3.3 节，第 349 页）。在 `lambda` 表达式产生的类中含有一个重载的函数调用运算符，例如，对于我们传递给 `stable_sort` 作为其最后一个实参的 `lambda` 表达式来说：

```
// 根据单词的长度对其进行排序，对于长度相同的单词按照字母表顺序排序
stable_sort(words.begin(), words.end(),
            [](const string &a, const string &b)
            { return a.size() < b.size(); });
```

其行为类似于下面这个类的一个未命名对象

```
class ShorterString {
public:
    bool operator()(const string &s1, const string &s2) const
    { return s1.size() < s2.size(); }
};
```

产生的类只有一个函数调用运算符成员，它负责接受两个 `string` 并比较它们的长度，它的形参列表和函数体与 `lambda` 表达式完全一样。如我们在 10.3.3 节（第 352 页）所见，默认情况下 `lambda` 不能改变它捕获的变量。因此在默认情况下，由 `lambda` 产生的类当中的函数调用运算符是一个 `const` 成员函数。如果 `lambda` 被声明为可变的，则调用运算符就不是 `const` 的了。

用这个类替代 `lambda` 表达式后，我们可以重写并重新调用 `stable_sort`：

```
stable_sort(words.begin(), words.end(), ShorterString());
```

第三个实参是新构建的 `ShorterString` 对象，当 `stable_sort` 内部的代码每次比较两个 `string` 时就会“调用”这一对象，此时该对象将调用运算符的函数体，判断第一个 `string` 的大小小于第二个时返回 `true`。

表示 `lambda` 及相应捕获行为的类

如我们所知，当一个 `lambda` 表达式通过引用捕获变量时，将由程序负责确保 `lambda` 执行时引用所引的对象确实存在（参见 10.3.3 节，第 350 页）。因此，编译器可以直接使用该引用而无须在 `lambda` 产生的类中将其存储为数据成员。

相反，通过值捕获的变量被拷贝到 `lambda` 中（参见 10.3.3 节，第 350 页）。因此，这种 `lambda` 产生的类必须为每个值捕获的变量建立对应的数据成员，同时创建构造函数，令其使用捕获的变量的值来初始化数据成员。举个例子，在 10.3.2 节（第 347 页）中有一个 `lambda`，它的作用是找到第一个长度不小于给定值的 `string` 对象：

```
// 获得第一个指向满足条件元素的迭代器，该元素满足 size() is >= sz
auto wc = find_if(words.begin(), words.end(),
                 [sz](const string &a)
                 { return a.size() >= sz; });
```

该 `lambda` 表达式产生的类将形如：

```
class SizeComp {
    SizeComp(size_t n): sz(n) { } // 该形参对应捕获的变量
    // 该调用运算符的返回类型、形参和函数体都与 lambda 一致
    bool operator()(const string &s) const
    { return s.size() >= sz; }
```

```
private:
    size_t sz; // 该数据成员对应通过值捕获的变量
};
```

和我们的 `ShorterString` 类不同，上面这个类含有一个数据成员以及一个用于初始化该成员的构造函数。这个合成的类不含有默认构造函数，因此要想使用这个类必须提供一个实参：

```
// 获得第一个指向满足条件元素的迭代器，该元素满足 size() is >= sz
auto wc = find_if(words.begin(), words.end(), SizeComp(sz));
```

`lambda` 表达式产生的类不含默认构造函数、赋值运算符及默认析构函数；它是否含有默认的拷贝/移动构造函数则通常要视捕获的数据成员类型而定（参见 13.1.6 节，第 450 页和 13.6.2 节，第 475 页）。

14.8.1 节练习

练习 14.38: 编写一个类令其检查某个给定的 `string` 对象的长度是否与一个阈值相等。使用该对象编写程序，统计并报告在输入的文件中长度为 1 的单词有多少个、长度为 2 的单词有多少个、……、长度为 10 的单词又有多少个。

练习 14.39: 修改上一题的程序令其报告长度在 1 至 9 之间的单词有多少个、长度在 10 以上的单词又有多少个。

练习 14.40: 重新编写 10.3.2 节（第 349 页）的 `biggies` 函数，使用函数对象类替换其中的 `lambda` 表达式。

练习 14.41: 你认为 C++11 新标准为什么要增加 `lambda`？对于你自己来说，什么情况下会使用 `lambda`，什么情况下会使用类？

14.8.2 标准库定义的函数对象

标准库定义了一组表示算术运算符、关系运算符和逻辑运算符的类，每个类分别定义了一个执行命名操作的调用运算符。例如，`plus` 类定义了一个函数调用运算符用于对一对运算对象执行 + 的操作；`modulus` 类定义了一个调用运算符执行二元的 % 操作；`equal_to` 类执行 ==，等等。

这些类都被定义成模板的形式，我们可以为其指定具体的应用类型，这里的类型即调用运算符的形参类型。例如，`plus<string>` 令 `string` 加法运算符作用于 `string` 对象；`plus<int>` 的运算对象是 `int`；`plus<Sales_data>` 对 `Sales_data` 对象执行加法运算，以此类推：

```
plus<int> intAdd; // 可执行 int 加法的函数对
negate<int> intNegate; // 可对 int 值取反的函数对象
// 使用 intAdd::operator(int, int) 求 10 和 20 的和
int sum = intAdd(10, 20); // 等价于 sum = 30
sum = intNegate(intAdd(10, 20)); // 等价于 sum = 30
// 使用 intNegate::operator(int) 生成 -10
// 然后将 -10 作为 intAdd::operator(int, int) 的第二个参数
sum = intAdd(10, intNegate(10)); // sum = 0
```

◀ 575

表 14.2 所列的类型定义在 `functional` 头文件中。

表 14.2: 标准库函数对象		
算术	关系	逻辑
plus<Type>	equal_to<Type>	logical_and<Type>
minus<Type>	not_equal_to<Type>	logical_or<Type>
multiplies<Type>	greater<Type>	logical_not<Type>
divides<Type>	greater_equal<Type>	
modulus<Type>	less<Type>	
negate<Type>	less_equal<Type>	

在算法中使用标准库函数对象

表示运算符的函数对象类常用来替换算法中的默认运算符。如我们所知，在默认情况下排序算法使用 `operator<` 将序列按照升序排列。如果要执行降序排列的话，我们可以传入一个 `greater` 类型的对象。该类将产生一个调用运算符并负责执行待排序类型的大于运算。例如，如果 `svec` 是一个 `vector<string>`，

```
// 传入一个临时的函数对象用于执行两个 string 对象的 > 比较运算
sort(svec.begin(), svec.end(), greater<string>());
```

则上面的语句将按照降序对 `svec` 进行排序。第三个实参是 `greater<string>` 类型的一个未命名的对象，因此当 `sort` 比较元素时，不再是使用默认的 `<` 运算符，而是调用给定的 `greater` 函数对象。该对象负责在 `string` 元素之间执行 `>` 比较运算。

需要特别注意的是，标准库规定其函数对象对于指针同样适用。我们之前曾经介绍过比较两个无关指针将产生未定义的行为（参见 3.5.3 节，第 107 页），然而我们可能会希望通过比较指针的内存地址来 `sort` 指针的 `vector`。直接这么做将产生未定义的行为，因此我们可以使用一个标准库函数对象来实现该目的：

```
vector<string *> nameTable; // 指针的 vector
// 错误：nameTable 中的指针彼此之间没有关系，所以 < 将产生未定义的行为
sort(nameTable.begin(), nameTable.end(),
    [](string *a, string *b) { return a < b; });
// 正确：标准库规定指针的 less 是定义良好的
sort(nameTable.begin(), nameTable.end(), less<string*>());
```

576

关联容器使用 `less<key_type>` 对元素排序，因此我们可以定义一个指针的 `set` 或者在 `map` 中使用指针作为关键值而无须直接声明 `less`。

14.8.2 节练习

练习 14.42: 使用标准库函数对象及适配器定义一条表达式，令其

- (a) 统计大于 1024 的值有多少个。
- (b) 找到第一个不等于 `pooh` 的字符串。
- (c) 将所有的值乘以 2。

练习 14.43: 使用标准库函数对象判断一个给定的 `int` 值是否能被 `int` 容器中的所有元素整除。

14.8.3 可调用对象与 function

C++语言中有几种可调用的对象：函数、函数指针、lambda 表达式（参见 10.3.2 节，第 346 页）、bind 创建的对象（参见 10.3.4 节，第 354 页）以及重载了函数调用运算符的类。

和其他对象一样，可调用的对象也有类型。例如，每个 lambda 有它自己唯一的（未命名）类类型；函数及函数指针的类型则由其返回值类型和实参类型决定，等等。

然而，两个不同类型的可调用对象却可能共享同一种调用形式（call signature）。调用形式指明了调用返回的类型以及传递给调用的实参类型。一种调用形式对应一个函数类型，例如：

```
int(int, int)
```

是一个函数类型，它接受两个 int、返回一个 int。

不同类型可能具有相同的调用形式

对于几个可调用对象共享同一种调用形式的情况，有时我们会希望把它们看成具有相同的类型。例如，考虑下列不同类型的可调用对象：

```
// 普通函数
int add(int i, int j) { return i + j; }
// lambda，其产生一个未命名的函数对象类
auto mod = [](int i, int j) { return i % j; };
// 函数对象类
struct divide {
    int operator()(int denominator, int divisor) {
        return denominator / divisor;
    }
};
```

上面这些可调用对象分别对其参数执行了不同的算术运算，尽管它们的类型各不相同，但是共享同一种调用形式： ◀ 577

```
int(int, int)
```

我们可能希望使用这些可调用对象构建一个简单的桌面计算器。为了实现这一目的，需要定义一个函数表（function table）用于存储指向这些可调用对象的“指针”。当程序需要执行某个特定的操作时，从表中查找该调用的函数。

在 C++语言中，函数表很容易通过 map 来实现。对于此例来说，我们使用一个表示运算符符号的 string 对象作为关键字；使用实现运算符的函数作为值。当我们要求给定运算符的值时，先通过运算符索引 map，然后调用找到的那个元素。

假定我们的所有函数都相互独立，并且只处理关于 int 的二元运算，则 map 可以定义成如下的形式：

```
// 构建从运算符到函数指针的映射关系，其中函数接受两个 int、返回一个 int
map<string, int(*) (int,int)> binops;
```

我们可以按照下面的形式将 add 的指针添加到 binops 中：

```
// 正确：add 是一个指向正确类型函数的指针
binops.insert({"+", add}); // {"+", add} 是一个 pair（参见 11.2.3 节，379 页）
```

但是我们不能将 mod 或者 divide 存入 binops：

```
binops.insert({"%", mod}); // 错误: mod 不是一个函数指针
```

问题在于 mod 是个 lambda 表达式，而每个 lambda 有它自己的类类型，该类型与存储在 binops 中的值的类型不匹配。

标准库 function 类型

C++
11

我们可以使用一个名为 **function** 的新的标准库类型解决上述问题，function 定义在 functional 头文件中，表 14.3 列举出了 function 定义的操作。

表 14.3: function 的操作	
function<T> f;	f 是一个用来存储可调用对象的空 function，这些可调用对象的调用形式应该与函数类型 T 相同（即 T 是 <i>retType(args)</i> ）
function<T> f(nullptr);	显式地构造一个空 function
function<T> f(obj);	在 f 中存储可调用对象 obj 的副本
f	将 f 作为条件：当 f 含有一个可调用对象时为真；否则为假
f(args)	调用 f 中的对象，参数是 args
定义为 function<T> 的成员的类型	
result_type	该 function 类型的可调用对象返回的类型
argument_type	当 T 有一个或两个实参时定义的类型。如果 T 只有一个实参，则 argument_type 是该类型的同义词；如果 T 有两个实参，则 first_argument_type 和 second_argument_type 分别代表两个实参的类型
first_argument_type	
second_argument_type	

function 是一个模板，和我们使用过的其他模板一样，当创建一个具体的 function 类型时我们必须提供额外的信息。在此例中，所谓额外的信息是指该 function 类型能够表示的对象的调用形式。参考其他模板，我们在一对尖括号内指定类型：

```
function<int(int, int)>
```

在这里我们声明了一个 function 类型，它可以表示接受两个 int、返回一个 int 的可调用对象。因此，我们可以用这个新声明的类型表示任意一种桌面计算器用到的类型；

```
function<int(int, int)> f1 = add; // 函数指针
function<int(int, int)> f2 = divide(); // 函数对象类的对象
function<int(int, int)> f3 = [](int i, int j) // lambda
    { return i * j; };
cout << f1(4,2) << endl; // 打印 6
cout << f2(4,2) << endl; // 打印 2
cout << f3(4,2) << endl; // 打印 8
```

578

使用这个 function 类型我们可以重新定义 map：

```
// 列举了可调用对象与二元运算符对应关系的表格
// 所有可调用对象都必须接受两个 int、返回一个 int
// 其中的元素可以是函数指针、函数对象或者 lambda
map<string, function<int(int, int)>> binops;
```

我们能把所有可调用对象，包括函数指针、lambda 或者函数对象在内，都添加到这个 map 中：

```
map<string, function<int(int, int)>> binops = {
    {"+", add}, // 函数指针
    {"-", std::minus<int>()}, // 标准库函数对象
    {"/", divide()}, // 用户定义的函数对象
    {"*", [](int i, int j) { return i * j; }}, // 未命名的 lambda
    {"%", mod} }; // 命名了的 lambda 对象
```

我们的 `map` 中包含 5 个元素，尽管其中的可调用对象的类型各不相同，我们仍然能够把所有这些类型都存储在同一个 `function<int (int , int)>` 类型中。

一如往常，当我们索引 `map` 时将得到关联值的一个引用。如果我们索引 `binops`，将得到 `function` 对象的引用。`function` 类型重载了调用运算符，该运算符接受它自己的实参然后将其传递给存好的可调用对象：

```
binops["+"](10, 5); // 调用 add(10, 5)
binops["-"](10, 5); // 使用 minus<int>对象的调用运算符
binops["/"](10, 5); // 使用 divide 对象的调用运算符
binops["*"](10, 5); // 调用 lambda 函数对象
binops["%"](10, 5); // 调用 lambda 函数对象
```

我们依次调用了 `binops` 中存储的每个操作。在第一个调用中，我们获得的元素存放着一个指向 `add` 函数的指针，因此调用 `binops["+"](10, 5)` 实际上是使用该指针调用 `add`，并传入 10 和 5。在接下来的调用中，`binops["-"]` 返回一个存放着 `std::minus<int>` 类型对象的 `function`，我们将执行该对象的调用运算符。

重载的函数与 function

我们不能（直接）将重载函数的名字存入 `function` 类型的对象中：

```
int add(int i, int j) { return i + j; }
Sales_data add(const Sales_data&, const Sales_data&);
map<string, function<int(int, int)>> binops;
binops.insert( {"+", add} ); // 错误：哪个 add？
```

解决上述二义性问题的一条途径是存储函数指针（参见 6.7 节，第 221 页）而非函数的名字：

```
int (*fp)(int,int) = add; // 指针所指的 add 是接受两个 int 的版本
binops.insert( {"+", fp} ); // 正确：fp 指向一个正确的 add 版本
```

同样，我们也能使用 `lambda` 来消除二义性：

```
// 正确：使用 lambda 来指定我们希望使用的 add 版本
binops.insert( {"+", [](int a, int b) {return add(a, b);} } );
```

`lambda` 内部的函数调用传入了两个 `int`，因此该调用只能匹配接受两个 `int` 的 `add` 版本，而这也正是执行 `lambda` 时真正调用的函数。



新版本标准库中的 `function` 类与旧版本中的 `unary_function` 和 `binary_function` 没有关联，后两个类已经被更通用的 `bind` 函数替代了（参见 10.3.4 节，第 357 页）。

14.8.3 节练习

练习 14.44: 编写一个简单的桌面计算器使其能处理二元运算。



14.9 重载、类型转换与运算符

在 7.5.4 节（第 263 页）中我们看到由一个实参调用的非显式构造函数定义了一种隐式的类型转换，这种构造函数将实参类型的对象转换成类类型。我们同样能定义对于类类型的类型转换，通过定义类型转换运算符可以做到这一点。转换构造函数和类型转换运算符共同定义了类类型转换（class-type conversions），这样的转换有时也被称作用户定义的类型转换（user-defined conversions）。

14.9.1 类型转换运算符

类型转换运算符（conversion operator）是类的一种特殊成员函数，它负责将一个类类型的值转换成其他类型。类型转换函数的一般形式如下所示：

```
operator type() const;
```

其中 *type* 表示某种类型。类型转换运算符可以面向任意类型（除了 `void` 之外）进行定义，只要该类型能作为函数的返回类型（参见 6.1 节，第 184 页）。因此，我们不允许转换成数组或者函数类型，但允许转换成指针（包括数组指针及函数指针）或者引用类型。

类型转换运算符既没有显式的返回类型，也没有形参，而且必须定义成类的成员函数。类型转换运算符通常不应该改变待转换对象的内容，因此，类型转换运算符一般被定义成 `const` 成员。



一个类型转换函数必须是类的成员函数；它不能声明返回类型，形参列表也必须为空。类型转换函数通常应该是 `const`。

定义含有类型转换运算符的类

举个例子，我们定义一个比较简单的类，令其表示 0 到 255 之间的一个整数：

```
class SmallInt {
public:
    SmallInt(int i = 0): val(i)
    {
        if (i < 0 || i > 255)
            throw std::out_of_range("Bad SmallInt value");
    }
    operator int() const { return val; }
private:
    std::size_t val;
};
```

我们的 `SmallInt` 类既定义了向类类型的转换，也定义了从类类型向其他类型的转换。其中，构造函数将算术类型的值转换成 `SmallInt` 对象，而类型转换运算符将 `SmallInt` 对象转换成 `int`：

```
SmallInt si;
```